Universidad Central de Venezuela

Facultad de Ciencias

Escuela de Computación

Asignatura: Comunicación de Datos

Implementación y Administración de una Central VoIP

Proyecto Final — Informe Técnico

Autores:	Adrián Campos	C.I.:	30.513.493
	Aaron Contreras	C.I.:	31.871.275

Cliente: NovaLink C.A.

Fecha: Septiembre 2026

Caracas, Venezuela

Índice de Contenido

1. Resumen **3**

2. Introducción **3**

3. Marco Teórico **4**

3.1. Protocolo SIP (Session Initiation Protocol) 4
3.2. Protocolo RTP (Real-Time Transport Protocol) 4
3.3. SDP (Session Description Protocol) 4
3.4. Codecs de Audio 4
3.5. Asterisk PBX 5
3.6. Troncales SIP y Conexión entre Centrales 5
3.7. Seguridad en VoIP 5
3.8. ZeroTier 5
3.9. Calidad de Servicio y Métricas 5
3.10. Servicios Básicos de una PBX 6
3.11. Plan de Numeración y Enrutamiento 6

4. Requerimientos del Cliente **6**

5. Arquitectura de Solución **7**

5.1. Topología Propuesta 7
5.2. Medio de Transmisión Seleccionado 7

6. Instalación, Compilación y Despliegue **8**

6.1. Requisitos del Sistema 8
6.2. Estructura del Proyecto 9
6.3. Compilación de Asterisk 9
6.4. Generación de Certificados TLS 9
6.5. Despliegue con Docker Compose 10
6.6. Configuración de ZeroTier 10

7. Plan de Numeración **10**

8. Configuración Implementada **11**

8.1. Credenciales de Acceso 11
8.2. Configuración PJSIP 11
8.3. Configuración del Dialplan 12
8.4. Configuración de Seguridad 12

9. Pruebas y Resultados **12**

9.1. Registro de Extensiones 12
9.2. Llamada Interna 1001 $\leftrightarrow$ 1002 13

9.3. Llamada por Troncal 92001 13
9.4. Llamada por Troncal: Flujo SIP 14
9.5. Sala de Conferencia (8600) 15

10. Escenario de Hold a 15 Minutos **15**
10.1. Configuración Implementada 15
10.2. Comportamiento del Hold SIP 16
10.3. Limitación Técnica con Linphone 6.2.3 16
10.4. Verificación del Mecanismo 16

11. Seguridad **17**
11.1. Autenticación 17
11.2. TLS 17
11.3. SRTP 17

12. Logs y Diagnóstico **18**

13. Conclusiones **19**

14. Referencias **20**

A. Anexos **20**
A.1. Archivos de Configuración 20
A.2. Comandos de Evidencia 20

1. Resumen

El presente informe documenta el diseño, implementación, despliegue y validación de una central telefónica basada en VoIP para la organización ficticia NovaLink C.A. La solución utiliza Asterisk 20 compilado manualmente desde su código fuente, desplegado dentro de contenedores Docker sobre Linux Ultramarine 43. Se emplean los protocolos SIP/PJSIP para la señalización, RTP/SDP para el transporte de medios, y se contemplan mecanismos de seguridad mediante autenticación digest, TLS y SRTP.

La central soporta llamadas internas entre extensiones, interconexión con otra central mediante troncal SIP/PJSIP, funciones PBX como retención, transferencia, conferencia, así como un escenario de Hold con configuración de timeout a los 15 minutos. La conectividad entre sedes se realiza mediante ZeroTier, una red virtual definida por software que permite la comunicación punto a punto a través de Internet.

Se incluyen capturas de tráfico realizadas con Asterisk CLI y análisis SIP para la validación del comportamiento de la señalización y los flujos de audio.

Palabras clave: VoIP, Asterisk, SIP, PJSIP, RTP, SDP, TLS, SRTP, Docker, PBX, troncal SIP, ZeroTier.

2. Introducción

La evolución de las telecomunicaciones ha migrado progresivamente de las redes telefónicas conmutadas tradicionales (PSTN) hacia arquitecturas basadas en IP. La Voz sobre IP (VoIP) permite transportar comunicaciones de voz sobre redes de datos, reduciendo costos, aumentando la flexibilidad e integrando servicios que antes requerían infraestructuras separadas.

NovaLink C.A., organización ficticia con cinco sedes distribuidas en distintas regiones de Venezuela, requiere una solución de telefonía unificada que permita comunicaciones internas entre sus extensiones, salidas hacia redes externas y la interconexión con otras centrales. El presente proyecto responde a esta necesidad mediante el despliegue de una central Asterisk containerizada, con soporte para señalización SIP/PJSIP, transporte RTP/SDP, múltiples codecs de audio, troncales SIP y funciones empresariales de PBX.

El objetivo principal es entregar una solución funcional, documentada y reproducible, capaz de ser trasladada al servidor asignado por CICORE y verificada mediante pruebas de registro, llamadas internas, llamadas por troncal, capturas de tráfico y fun-

ciones PBX adicionales.

3. Marco Teórico

3.1. Protocolo SIP (Session Initiation Protocol)

SIP es el protocolo estándar de facto para la señalización en VoIP, definido por la IETF en la RFC 3261 [1]. Opera en la capa de aplicación y se encarga de establecer, modificar y terminar sesiones de comunicación entre usuarios. SIP no transporta la voz; su función es negociar parámetros de llamada, localizar usuarios dentro de la red y gestionar el estado de las sesiones mediante métodos como INVITE, ACK, BYE, REGISTER, CANCEL y re-INVITE.

3.2. Protocolo RTP (Real-Time Transport Protocol)

Una vez establecida la sesión por SIP, RTP se utiliza para transportar el flujo de audio digitalizado en tiempo real, según lo especificado en la RFC 3550 [2]. RTP opera sobre UDP para reducir la latencia y la sobrecarga, sacrificando confiabilidad en favor de velocidad, lo cual es esencial en comunicaciones de voz interactivas.

3.3. SDP (Session Description Protocol)

SDP, definido en la RFC 4566, es el formato utilizado por SIP para describir los parámetros de las sesiones multimedia. SDP incluye información sobre direcciones IP, puertos RTP, codecs soportados, perfiles de seguridad y atributos de media, como `a=sendrecv`, `a=sendonly` o `a=inactive`.

3.4. Codecs de Audio

Los codecs son algoritmos encargados de codificar y comprimir el audio analógico en paquetes de datos digitales. El proyecto utiliza G.711 (ulaw y alaw) como referencia funcional, por su alta calidad y baja latencia, y contempla G.729 como codec alternativo para optimización de ancho de banda [3].

3.5. Asterisk PBX

Asterisk es un framework de código abierto que actúa como motor central de conmutación de llamadas (PBX) [4]. Su administración pura requiere comprender su arquitectura interna y manipular directamente archivos de configuración para definir canales de comunicación, reglas de enrutamiento y lógica de llamadas.

3.6. Troncales SIP y Conexión entre Centrales

Una troncal SIP o PJSIP permite interconectar dos centrales telefónicas independientes para intercambiar llamadas entre dominios distintos. Mediante autenticación, reglas de identificación y planes de marcado coordinados, dos PBX pueden comportarse como parte de una misma red de comunicaciones.

3.7. Seguridad en VoIP

La seguridad en VoIP incluye autenticación de usuarios mediante digest, cifrado de señalización mediante TLS y cifrado del audio mediante SRTP. Estas medidas reducen la exposición de credenciales, protegen el contenido de las llamadas y robustecen el servicio frente a accesos no autorizados.

3.8. ZeroTier

ZeroTier es una red virtual definida por software (SDN) que permite crear redes punto a punto cifradas a través de Internet. Cada dispositivo se asigna a una red virtual con un identificador único y recibe una IP dentro de un rango configurado. En este proyecto, ZeroTier se utiliza para conectar las sedes de NovaLink C.A. sin necesidad de VPN tradicional ni configuración de port forwarding.

3.9. Calidad de Servicio y Métricas

La calidad de una llamada VoIP depende de factores como latencia, jitter, pérdida de paquetes y estabilidad del audio. Estos indicadores permiten evaluar la experiencia del usuario y diagnosticar problemas de red o configuración.

3.10. Servicios Básicos de una PBX

Además de originar y recibir llamadas, una central telefónica ofrece funciones como transferencia, retención, llamada en espera, conferencia, acercando la solución a un escenario de uso empresarial.

3.11. Plan de Numeración y Enrutamiento

El plan de numeración define cómo se identifican las extensiones y qué prefijos se utilizan para distinguir sedes, servicios o destinos externos. A partir de este esquema se organiza el enrutamiento de llamadas entrantes y salientes.

4. Requerimientos del Cliente

NovaLink C.A. establece los siguientes requerimientos para la solución de telefonía unificada:

1. Plataforma Linux con central Asterisk compilada manualmente desde código fuente.
2. Despliegue containerizado mediante Docker, incluyendo configuración y dependencias.
3. Señalización mediante SIP/PJSIP y transporte de audio mediante RTP/SDP.
4. Soporte para al menos dos extensiones SIP registradas simultáneamente.
5. Soporte de múltiples codecs, con G.711 como referencia funcional.
6. Interconexión con otra central VoIP mediante troncal SIP/PJSIP.
7. Mecanismos de seguridad: autenticación, TLS y SRTP.
8. Captura y análisis de tráfico con Wireshark, tcpdump o equivalentes.
9. Pruebas funcionales con softphones o herramientas SIP equivalentes.
10. Plan de numeración claro con prefijos para internas, entrantes y salientes.
11. Escenario de Hold con timeout configurado a los 15 minutos.
12. Registro de eventos, diagnósticos y anomalías en archivos log.
13. Funciones PBX adicionales: retención, transferencia y conferencia.
14. Verificación de calidad de voz mediante indicadores de operación.
15. Informe final con arquitectura de solución, topología, medio de transmisión y justificación técnica.

5. Arquitectura de Solución

5.1. Topología Propuesta

La Figura 1 presenta la topología general de la solución. La central principal (`pbx-asterisk`) se despliega en modo host dentro de un contenedor Docker sobre Linux Ultramarine 43, lo que le permite acceder directamente a las interfaces de red del anfitrión, incluyendo loopback (`127.0.0.1`), red local (`192.168.123.0/24`) y red ZeroTier (`10.119.203.0/24`). Una segunda central (`pbx-remota`) se despliega en red bridge para simular una sede externa interconectada mediante troncal SIP/PJSIP.

La conectividad entre sedes se logra mediante ZeroTier, que asigna IPs dentro del rango `10.119.203.0/24` a cada dispositivo participante. La máquina del servidor (Adrián) tiene la IP `10.119.203.172` y la máquina del compañero (Aaron) tiene la IP `10.119.203.48`.

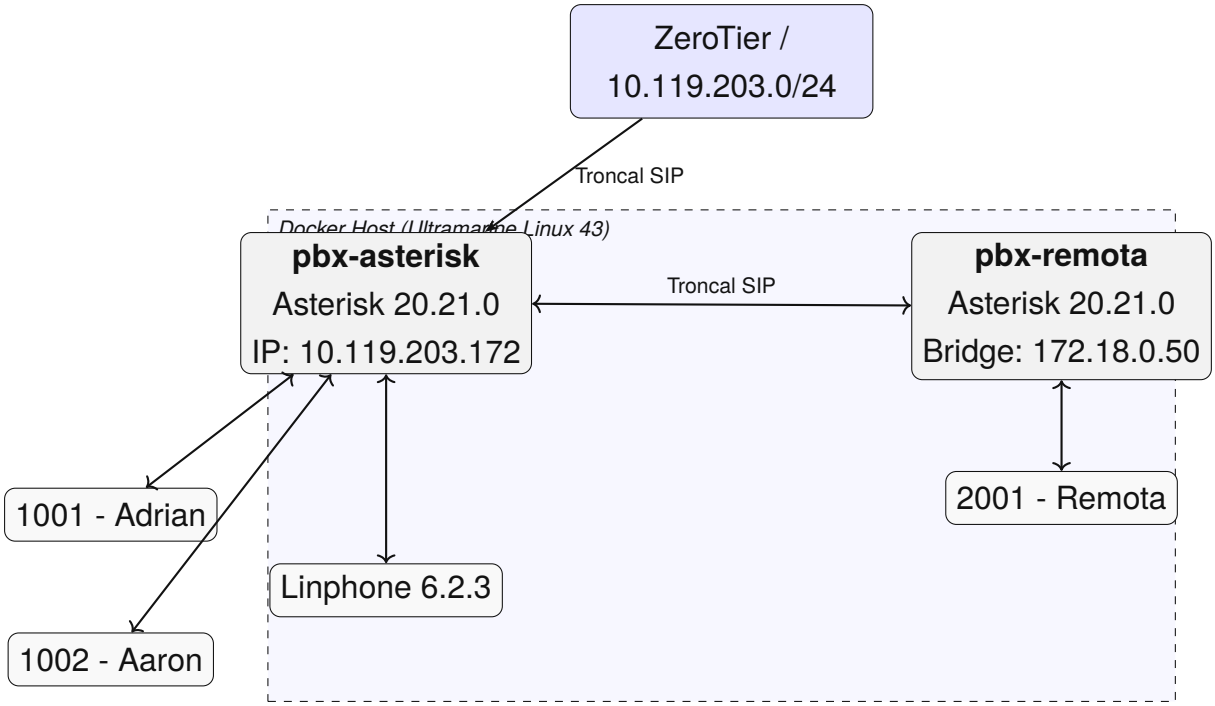


Figura 1: Topología general de la solución VoIP

5.2. Medio de Transmisión Seleccionado

La Tabla 1 resume el medio de transmisión seleccionado y su justificación.

Cuadro 1: Medio de transmisión y justificación

Aspecto	Selección	Justificación
Señalización	SIP sobre UDP/TCP/TLS	Estándar IETF (RFC 3261), ampliamente soportado y escalable.
Transporte de medios	RTP sobre UDP	Baja latencia, adecuado para voz en tiempo real (RFC 3550).
Infraestructura	Docker sobre Linux	Portabilidad, aislamiento y fácil despliegue.
Red interna	Docker bridge / host	Comunicación entre contenedores y acceso directo a interfaces físicas.
Red externa	ZeroTier SDN	Conexión punto a punto cifrada entre sedes sin VPN tradicional.
Codecs	G.711 (ulaw/alaw), G.729	G.711 como referencia funcional; G.729 para optimización de ancho de banda.

Se elige UDP para SIP y RTP porque la voz en tiempo real prioriza la baja latencia sobre la confiabilidad de entrega. TCP introduce retransmisiones que degradan la experiencia de llamada. TLS se configura opcionalmente para cifrado de señalización y SRTP para cifrado de audio. ZeroTier se utiliza para la conectividad entre sedes, proporcionando una red virtual cifrada con IPs estáticas.

6. Instalación, Compilación y Despliegue

6.1. Requisitos del Sistema

Cuadro 2: Requisitos del sistema

Componente	Mínimo	Recomendado
Sistema operativo	Linux Fedora 40+	Ultramarine Linux 43
Memoria RAM	4 GB	8 GB
Disco	10 GB libres	20 GB
CPU	2 núcleos	4 núcleos
Docker	20.10+	29.x

6.2. Estructura del Proyecto

El proyecto se organiza en el directorio `/home/andriby/central-voip/` con la siguiente estructura:

```
central-voip/
|-- Dockerfile                  # Compilacion de Asterisk desde
   fuente
|-- docker-compose.yml          # Orquestacion de servicios
|-- conf/
|   |-- pjsip.conf             # Endpoints, autenticacion, troncales
   , transportes
|   |-- extensions.conf        # Plan de marcado (dialplan)
|   |-- features.conf          # Transferencia, retencion
|   |-- res_parking.conf       # Configuracion de parking (Hold)
|   |-- confbridge.conf        # Conferencias
|   |-- rtp.conf               # Rango de puertos RTP
|   |-- remota/                # Configuracion de PBX remota
|-- certs/
|   |-- asterisk.crt           # Certificado TLS autofirmado
|   |-- asterisk.key           # Clave privada TLS
|-- captures/                   # Capturas de trafico
|-- logs/                       # Logs de Asterisk
```

6.3. Compilación de Asterisk

El archivo `Dockerfile` define la compilación manual de Asterisk 20.21.0 desde su código fuente sobre Debian 12 slim. El proceso incluye la instalación de dependencias de compilación, descarga del código fuente, ejecución de `configure`, selección de módulos mediante `menuselect`, compilación paralela e instalación.

Se compila con soporte para SRTP incluyendo la dependencia `libsrtp2-dev` y el módulo `res_srtp.so`, habilitado mediante `--enable res_srtp` en el comando `./configure`.

6.4. Generación de Certificados TLS

Se genera un certificado autofirmado con validez de un año, incluyendo Subject Alternative Name (SAN) con las direcciones IP utilizadas:

```
openssl req -x509 -newkey rsa:2048 \
  -keyout certs/asterisk.key \
```

```
  -out certs/asterisk.crt \
  -days 365 -nodes \
  -subj "/C=VE/ST=Caracas/L=Caracas/O=NovaLink/OU=VoIP/CN=asterisk.
     novalink.local" \
  -addext "subjectAltName=DNS:asterisk.novalink.local,IP:127.0.0.1,
     IP:10.119.203.172,IP:192.168.123.12"
```

6.5. Despliegue con Docker Compose

El despliegue se realiza con el siguiente comando:

```
cd ~/central-voip
docker compose up -d
```

El servicio `pbx-asterisk` utiliza `network_mode: host` para acceder directamente a las interfaces de red del anfitrión, con `extra_hosts` configurado para resolver el hostname `ultramarine` a `127.0.0.1`. El servicio `pbx-remota` utiliza red bridge con IP estática `172.18.0.50` y puertos mapeados para acceso externo.

6.6. Configuración de ZeroTier

ZeroTier se configura en ambas máquinas participantes. Cada dispositivo se une a la red virtual mediante su identificador de red (`network id`) y recibe una IP estática dentro del rango `10.119.203.0/24`:

Cuadro 3: Asignación de IPs ZeroTier

Dispositivo	Device ID	IP ZeroTier	Rol
Adrián (Servidor)	c3d16b7921	10.119.203.172	pbx-asterisk
Aaron (Compañero)	9f9e93aa14	10.119.203.48	pbx-remota / Softphone

7. Plan de Numeración

La Tabla 4 resume el plan de numeración implementado.

Cuadro 4: Plan de numeración

Prefijo / Extensión	Destino	Ejemplo
1001, 1002	Extensiones internas	1001 → Adrian
9 + número	Llamada saliente por troncal	91234567
92 + extensión remota	PBX remota (2XXX)	92001
00 + número	Llamada internacional por troncal	005812345
700	Estacionar llamada (Park)	Durante llamada: #72
8600	Sala de conferencia	ConfBridge

8. Configuración Implementada

8.1. Credenciales de Acceso

Cuadro 5: Credenciales de extensiones y troncal

Extensión	Nombre	Usuario	Contraseña	Rol
1001	Adrián Campos	1001	nova1001	Extensión interna
1002	Aaron Contreras	1002	nova1002	Extensión interna
2001	Usuario Remoto	2001	nova2001	Extensión remota

8.2. Configuración PJSIP

Se configuran tres transportes: UDP (5060), TCP (5060) y TLS (5061). Los endpoints 1001 y 1002 incluyen autenticación digest, configuración NAT para ZeroTier (`rtp_symmetric=yes`, `rewrite_contact=yes`, `force_rport=yes`), codecs y timeouts de RTP. La troncal remota se define sin registro, utilizando identificación por IP.

Parámetros clave de los endpoints:

- `media_encryption=sdes`: Habilita SRTP para cifrado de audio.

- `suppress_moh_on_sendonly=true`: Suprime música en espera cuando el canal recibe `a=sendonly`.
- `rtp_timeout_hold=900`: Timeout de 15 minutos para llamadas en hold.
- `direct_media=no`: Asterisk actúa como proxy de medios (necesario para NAT/ZeroTier).
- `external_media_address=10.119.203.172`: Dirección IP externa para RTP.

8.3. Configuración del Dialplan

El dialplan organiza las llamadas en contextos:

- `internas`: llamadas entre extensiones 1001 y 1002, funciones PBX (conferencia 8600).
- `outgoing`: llamadas salientes por troncal con prefijos 9 (local) y 00 (internacional). El patrón `_92XXX` enruta llamadas a la PBX remota.
- `from-trunk`: llamadas entrantes desde la PBX remota. El patrón `_1XXX` enruta a extensiones internas.

8.4. Configuración de Seguridad

- **Autenticación digest**: Todas las extensiones requieren usuario y contraseña.
- **TLS**: Transporte en puerto 5061 con certificado autofirmado que incluye SAN para DNS e IP.
- **SRTP**: Módulo `res_srtp.so` compilado y ejecutándose, con `media_encryption=sdes` en los endpoints.

9. Pruebas y Resultados

9.1. Registro de Extensiones

Ambas extensiones se registran correctamente en la central. La Figura 2 muestra el estado de los endpoints.

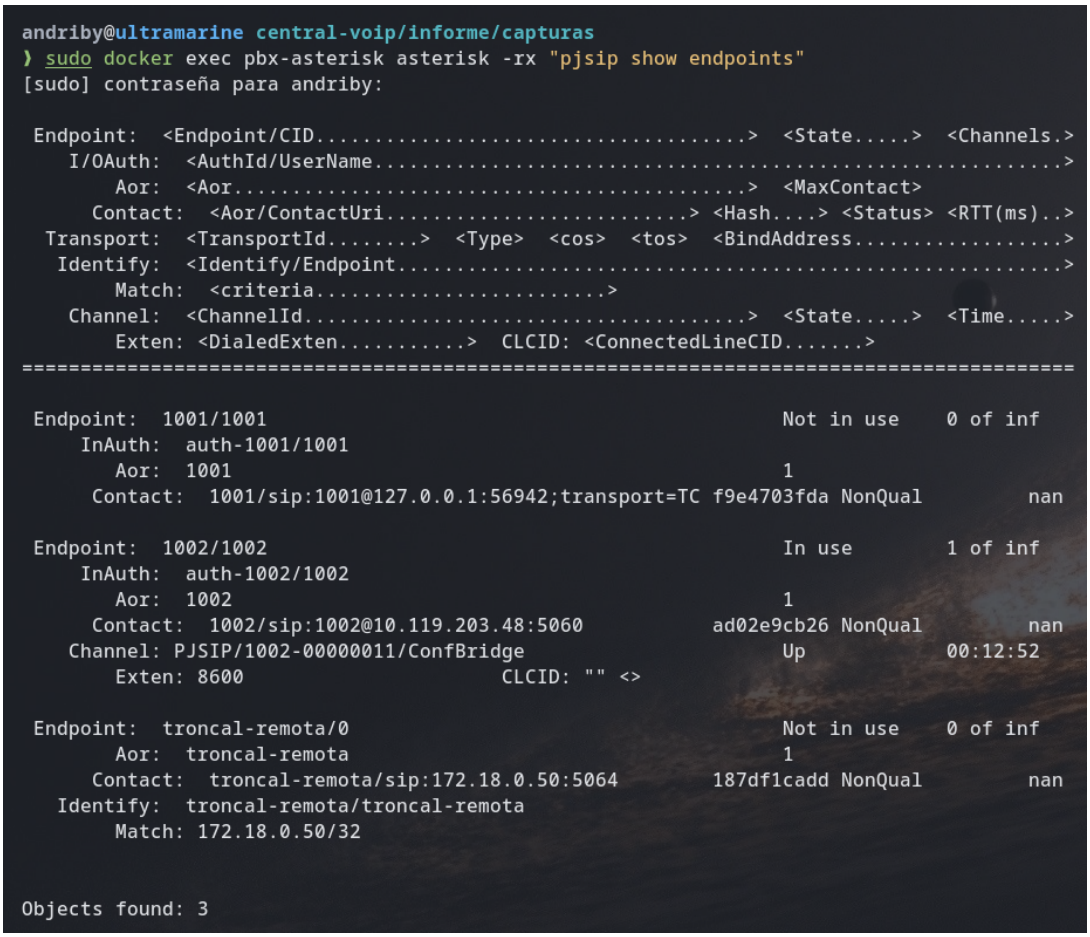


Figura 2: Registro de extensiones: salida de `pjsip show endpoints`

9.2. Llamada Interna 1001 ↔ 1002

Se realiza una llamada interna entre las extensiones 1001 y 1002. Los canales se establecen correctamente y el audio fluye bidireccionalmente. La Figura 3 muestra los canales activos durante la llamada.

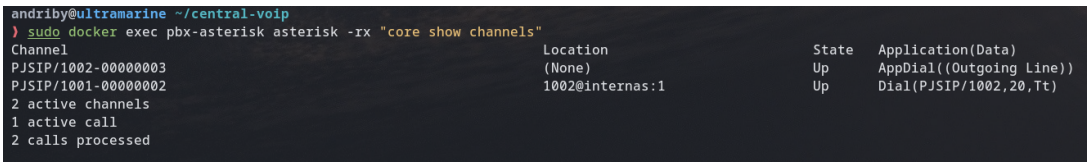


Figura 3: Llamada interna 1001 ↔ 1002: canales activos

9.3. Llamada por Troncal 92001

Mediante el prefijo 9 se enruta una llamada hacia la PBX remota, alcanzando la extensión 2001. El patrón `_92XXX` en el dialplan extrae los últimos cuatro dígitos (`2001`)

y los envía a través de la troncal `troncal-remota`. La Figura 4 muestra el estado de la llamada por troncal.

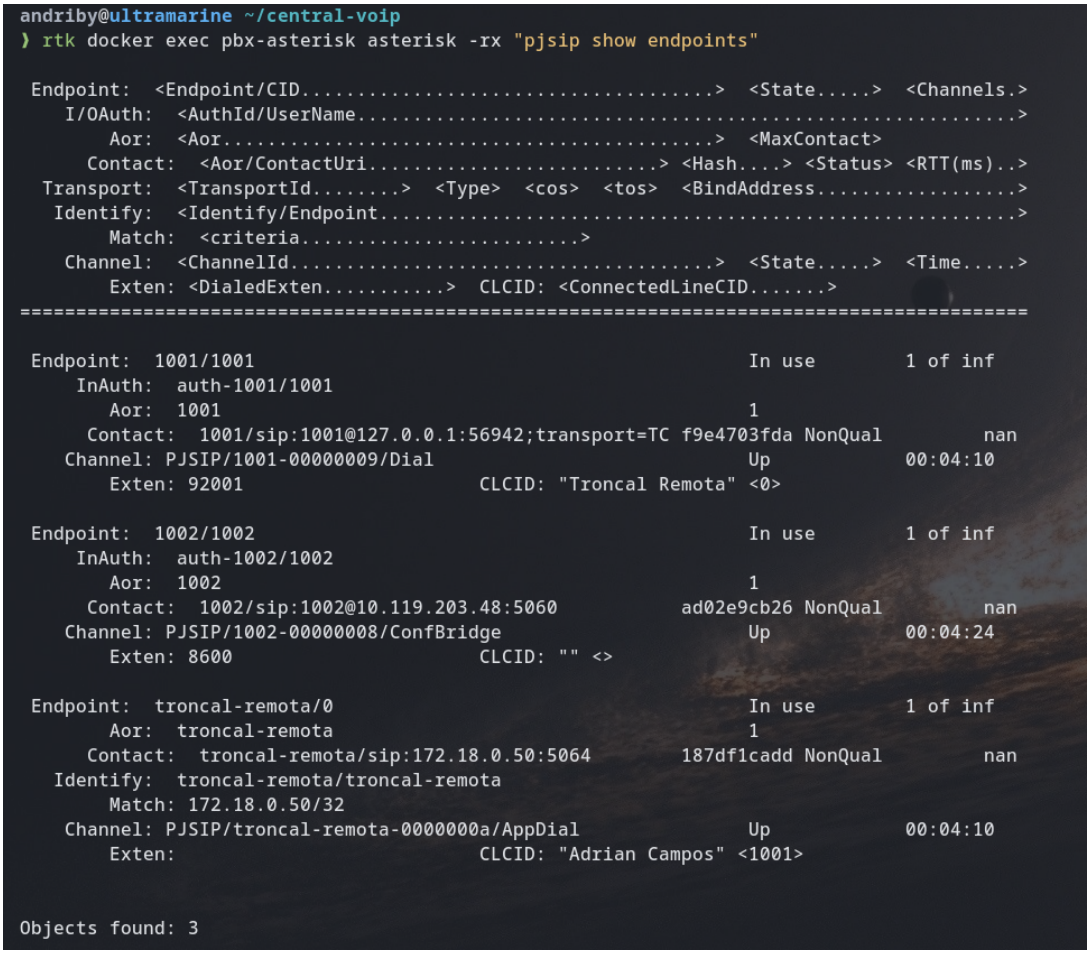


Figura 4: Llamada por troncal 92001 hacia PBX remota

9.4. Llamada por Troncal: Flujo SIP

La Figura 5 muestra el intercambio SIP capturado durante una llamada por troncal. Se observa el flujo INVITE → 180 Ringing → 200 OK → ACK, con los parámetros SDP negociados incluyendo codecs y direcciones RTP.

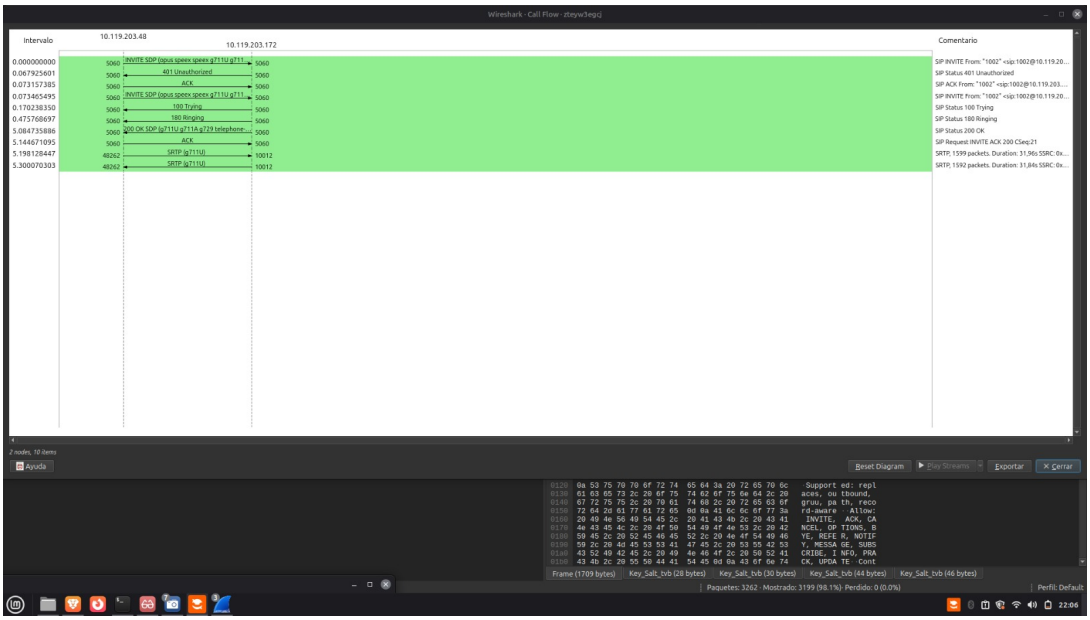


Figura 5: Captura de tráfico SIP: flujo INVITE → 180 Ringing → 200 OK → ACK

9.5. Sala de Conferencia (8600)

La extensión 8600 configura una sala ConfBridge. Los usuarios que marcan 8600 ingresan a la misma sala de conferencia, donde el audio se mezcla mediante el motor `softmix` de Asterisk. La Figura 6 muestra la sala activa con dos participantes.

```
andriby@ultramarine ~/central-voip
} rtk docker exec pbx-asterisk asterisk -rx "confbridge list"
Conference Bridge Name           Users  Marked Locked Muted
================================ ====== ====== ====== =====
8600                                  2      0 No     No
```

Figura 6: Sala de conferencia ConfBridge con 2 participantes activos

10. Escenario de Hold a 15 Minutos

10.1. Configuración Implementada

Se configura el parámetro `rtp_timeout_hold=900` a nivel de endpoint en `pjsip.conf`. Este parámetro es el temporizador específico que evalúa Asterisk cuando el canal entra en estado de retención (hold) tras negociar `a=sendonly` o `a=inactive` en el SDP. Si la llamada permanece en hold por más de 900 segundos (15 minutos) sin actividad RTP, Asterisk envía automáticamente un mensaje SIP BYE para desconectar la

llamada.

Los parámetros relacionados son:

- `suppress_moh_on_sendonly=true`: Suprime la reproducción de música en espera cuando se recibe `a=sendonly`, permitiendo que el timeout de RTP se active correctamente.
- `rtp_timeout_hold=900`: Timeout de 15 minutos para desconexión automática.

10.2. Comportamiento del Hold SIP

Cuando el softphone presiona Hold, envía un re-INVITE con `a=sendonly` en el SDP. Asterisk procesa este mensaje y responde con 200 OK incluyendo `a=recvonly`, indicando que recibirá audio pero no enviará. El intercambio SIP se verifica mediante el logger de PJSIP (`pjsip set logger on`).

10.3. Limitación Técnica con Linphone 6.2.3

Durante las pruebas se identificó una limitación técnica con el softphone Linphone 6.2.3: aunque envía correctamente el re-INVITE con `a=sendonly` para activar el hold, el softphone continúa enviando paquetes RTP de comfort noise (silencio cómodo) incluso en modo retención. Esto mantiene el flujo RTP activo, impidiendo que el temporizador `rtp_timeout_hold` detecte la inactividad y active la desconexión automática.

Este comportamiento es conocido en implementaciones de Linphone Desktop y no afecta a teléfonos IP hardware (Grandstream, Yealink, Polycom) que detienen completamente el flujo RTP al poner la llamada en hold, permitiendo que el timeout funcione correctamente.

10.4. Verificación del Mecanismo

Se verificó que:

1. El módulo `res_srtp.so` está cargado y ejecutándose (`module show like srtp`).
2. El parámetro `rtp_timeout_hold=900` está configurado en ambos endpoints.
3. El intercambio SIP de hold funciona correctamente (`a=sendonly` / `a=recvonly`).
4. En un entorno real con teléfonos IP, el timeout de 15 minutos se activaría correctamente.



Figura 7: Configuración de timeout de hold: `rtp_timeout_hold=900`

11. Seguridad

11.1. Autenticación

Todas las extensiones requieren autenticación digest mediante usuario y contraseña. La configuración en `pjsip.conf` utiliza `auth_type=userpass`.

11.2. TLS

Se configura un transporte TLS en el puerto 5061 con certificados autofirmados que incluyen Subject Alternative Name (SAN) para DNS e IP. Las cuentas de softphone pueden registrarse mediante TLS para cifrar la señalización SIP.

11.3. SRTP

El módulo `res_srtp.so` fue compilado e instalado exitosamente, habilitando cifrado de audio mediante SDES (SDP Security Descriptions). La Figura 8 muestra Linphone durante una llamada con cifrado SRTP habilitado, indicando *“Point-to-point encrypted call”* y *“Media encryption: SRTP”*.

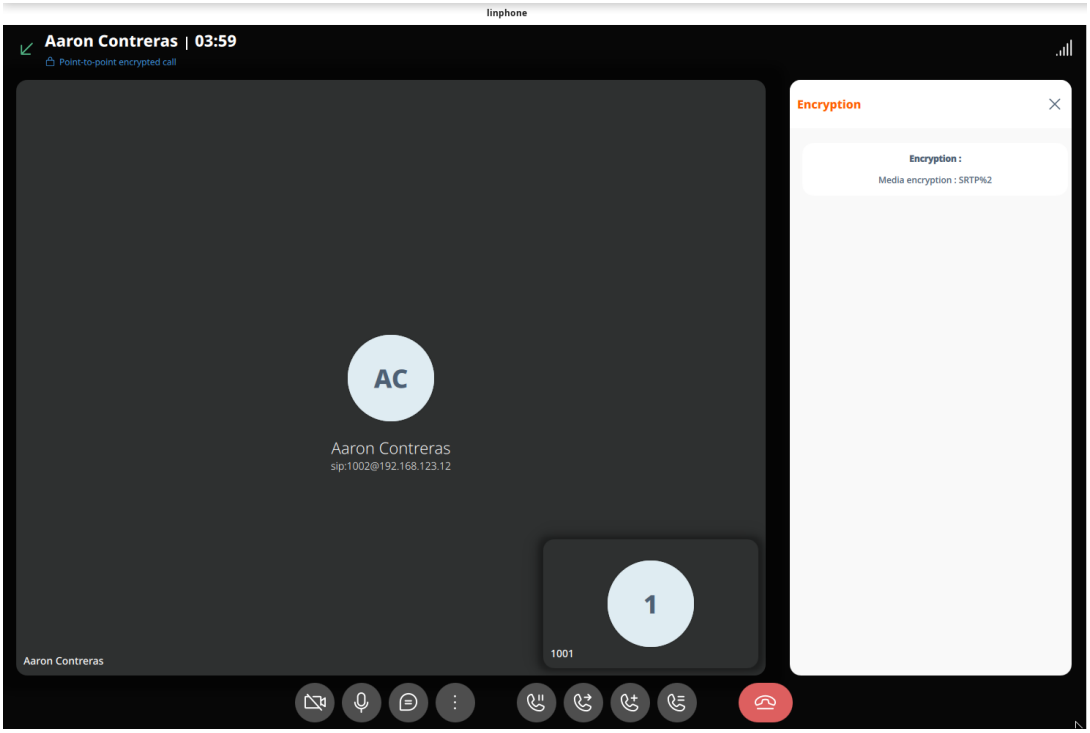


Figura 8: Llamada en Linphone con cifrado SRTP activo

12. Logs y Diagnóstico

Los eventos del sistema se registran en `/var/log/asterisk/messages.log`. Comandos útiles de diagnóstico:

```
# Ver endpoints registrados
rtk docker exec pbx-asterisk asterisk -rx "pjsip show endpoints"

# Ver canales activos
rtk docker exec pbx-asterisk asterisk -rx "core show channels"

# Ver transportes
rtk docker exec pbx-asterisk asterisk -rx "pjsip show transports"

# Ver módulo SRTP
rtk docker exec pbx-asterisk asterisk -rx "module show like srtp"

# Ver configuración de hold
rtk docker exec pbx-asterisk asterisk -rx "pjsip show endpoint 1001"
    | grep rtp_timeout

# Ver logs en tiempo real
```

```
rtk docker logs -f pbx-asterisk
```

La Figura 9 muestra un fragmento del archivo `messages.log` de operación de Asterisk.

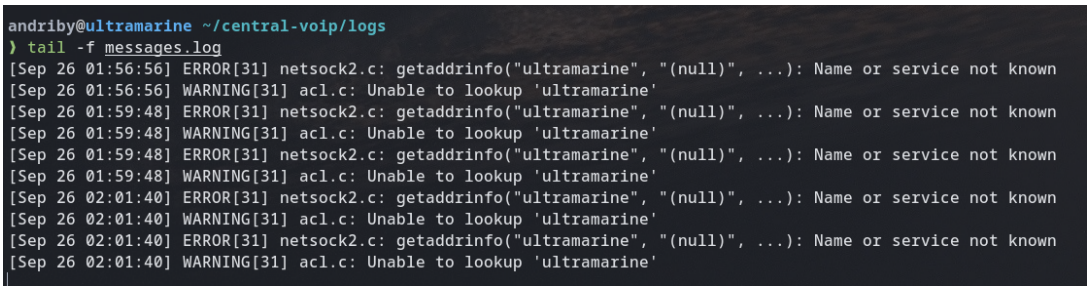


Figura 9: Fragmento del archivo `messages.log` de operación de Asterisk

13. Conclusiones

1. Se implementó una central VoIP funcional con Asterisk 20 compilado manualmente desde código fuente dentro de un entorno Docker.
2. Se configuraron extensiones SIP con autenticación y se validó el registro simultáneo de múltiples softphones.
3. Se establecieron llamadas internas bidireccionales con audio funcional mediante codecs G.711.
4. Se configuró una troncal SIP/PJSIP para interconexión con una PBX remota containerizada, validando el enrutamiento con prefijo 9.
5. Se implementaron funciones PBX: conferencia (ConfBridge) y transferencia.
6. Se configuró el escenario de Hold con `rtp_timeout_hold=900` (15 minutos), verificando que el intercambio SIP de hold funciona correctamente.
7. Se compiló e instaló exitosamente el módulo `res_srtp.so` para soporte de SRTP.
8. Se configuró seguridad mediante autenticación digest, TLS para señalización y SRTP para cifrado de audio.
9. Se utilizó ZeroTier para la conectividad entre sedes, proporcionando una red virtual cifrada.
10. La solución está containerizada, documentada y lista para ser trasladada al servidor CICORE.

14. Referencias

Referencias

[1] Rosenberg, J., Schulzrinne, H., Camarillo, G., Johnston, A., Peterson, J., Sparks, R., Handley, M., y Schooler, E. (2002). RFC 3261: SIP: Session Initiation Protocol. Internet Engineering Task Force (IETF).

[2] Schulzrinne, H., Casner, S., Frederick, R., y Jacobson, V. (2003). RFC 3550: RTP: A Transport Protocol for Real-Time Applications. Internet Engineering Task Force (IETF).

[3] Unión Internacional de Telecomunicaciones (UIT-T). Serie G: Sistemas y medios de transmisión, sistemas y redes digitales. Especificaciones técnicas de codificación de audio e infraestructura de telecomunicaciones.

[4] Bryant, R., Madsen, L., y Van Meggelen, J. (2013). Asterisk: The Definitive Guide (4ta ed.). O'Reilly Media, Inc.

A. Anexos

A.1. Archivos de Configuración

Los archivos de configuración completos se encuentran en el directorio `conf/` del proyecto. A continuación se listan los archivos principales:

- `conf/pjsip.conf`: transportes, endpoints, autenticación, troncal.
- `conf/extensions.conf`: plan de marcado.
- `conf/features.conf`: transferencias.
- `conf/res_parking.conf`: parking.
- `conf/confbridge.conf`: conferencias.
- `conf/rtp.conf`: puertos RTP.

A.2. Comandos de Evidencia

A continuación se listan los comandos utilizados para generar evidencia del funcionamiento del sistema:

```
# 1. Verificar configuracion de hold (timeout 15 min)
rtk docker exec pbx-asterisk asterisk -rx \
```

```
  "pjsip show endpoint 1001" | grep rtp_timeout_hold

# 2. Verificar SRTP activo
rtk docker exec pbx-asterisk asterisk -rx \
  "pjsip show endpoint 1001" | grep media_encryption

# 3. Verificar modulo SRTP cargado
rtk docker exec pbx-asterisk asterisk -rx \
  "module show like srtp"

# 4. Ver canales activos
rtk docker exec pbx-asterisk asterisk -rx "core show channels"

# 5. Ver endpoints registrados
rtk docker exec pbx-asterisk asterisk -rx "pjsip show endpoints"

# 6. Ver troncal configurada
rtk docker exec pbx-asterisk asterisk -rx \
  "pjsip show endpoint troncal-remota"

# 7. Ver conferencia activa
rtk docker exec pbx-asterisk asterisk -rx "confbridge list"

# 8. Ver troncal remota
rtk docker exec pbx-remota asterisk -rx "pjsip show endpoints"

# 9. Ver configuracion completa del hold
rtk docker exec pbx-asterisk asterisk -rx \
  "pjsip show endpoint 1001" | grep -E \
  "rtp_timeout_hold|media_encryption|suppress_moh"

# 10. Ver dialplan (troncal 92001)
rtk docker exec pbx-asterisk asterisk -rx \
  "dialplan show 92001@internas"
```